ISLAMIC UNIVERSITY OF TECHNOLOGY

Board Bazar, Gazipur, Dhaka.

Assignment : Understanding and Evaluating CPU Scheduling Algorithms in Modern Operating Systems
Operating Systems : CSE 4501

Student ID: 210042112

Fifth Semester
Department of Computer Science and Engineering
BSc. in Software Engineering

April 25, 2025

Contents

1 Assignment requirements

Provide a concise overview of CPU scheduling in operating systems. Discuss its role in managing process execution, optimizing resource utilization, and enhancing user experience. Define key concepts such as CPU burst, turnaround time, waiting time, and response time. Explain the distinction between preemptive and non-preemptive scheduling.

1.1 Core Topics and Tasks:

1.2 Understanding Scheduling Algorithms:

Discuss and explain the following CPU scheduling algorithms:

- First Come First Serve (FCFS)
- Shortest Job First (SJF)
- Priority Scheduling
- Round Robin (RR)
- Multilevel Queue
- Multilevel Feedback-Queue

For each:

- Provide a conceptual explanation and diagram.
- Highlight their advantages and disadvantages.
- Explain the scenarios/environments where each is most appropriate.

1.3 Algorithm Performance Evaluation:

Choose any three algorithms and compare them based on the following metrics:

- Average waiting time
- Turnaround time
- CPU utilization
- Fairness

Use hypothetical process data (at least 4–5 processes) to compute these values manually. Present the results in a comparative table and a graph for clarity.

1.4 Analytical Application:

- How does preemption influence system responsiveness and fairness?
- How does starvation occur in scheduling, and which algorithms are prone to it?
- Suggest and justify techniques (like aging) that address starvation.

1.5 Real-World OS Case Study Comparison:

Compare how two modern operating systems (e.g., Linux and Windows) implement CPU scheduling:

- Identify which algorithms are used (or hybrid models).
- Discuss how these systems prioritize processes and manage context switches.
- Analyze any differences in scheduling policies between desktop and server versions.

1.6 Visualization Task:

Design a Gantt chart for at least two algorithms based on your chosen set of processes from Task 2. Clearly show how CPU time is distributed among processes.

1.7 Critical Thinking Scenario:

You are designing an operating system for a real-time embedded system (like a pacemaker or automotive controller).

- Which CPU scheduling algorithm would you choose?
- Justify your choice in terms of reliability, determinism, and latency.

1.8 Conclusion:

Summarize your key learnings about CPU scheduling. Reflect on the role of scheduling in balancing performance, efficiency, and fairness in different computing environments.

2 Introduction

CPU scheduling is a fundamental function of an operating system (OS) that manages the execution of processes on the CPU. Its primary role is to ensure efficient resource utilization, fair process execution, and an optimal user experience by deciding which process runs next when the CPU becomes idle.

2.1 key objectives of CPU scheduling

1. **Maximizing CPU Utilization** – Keeping the CPU as busy as possible by reducing idle time.
2. **Improving Throughput** – Increasing the number of processes completed per unit time.
3. **Minimizing Turnaround, Waiting, and Response Times** – Enhancing process efficiency and user satisfaction.
4. **Ensuring Fairness** – Allocating CPU time fairly among all processes.

2.2 Short notes

1. **CPU Burst** – The time a process spends executing on the CPU before it switches to I/O operations.
2. **Turnaround Time** – Total time taken from process submission to completion (including waiting and execution time).
3. **Waiting Time** – Total time a process spends waiting in the ready queue.
4. **Response Time** – Time taken from process submission until the first response is produced (important in interactive systems).

2.3 Preemptive vs Non-preemptive scheduling

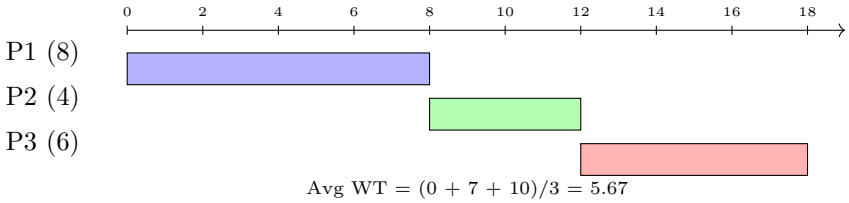
1. **Preemptive Scheduling** – The OS can interrupt a running process to allocate the CPU to a higher-priority process (e.g., Round Robin, SRTF).
    - pros: Better responsiveness, avoids CPU monopolization.
    - cons: Higher overhead due to frequent context switches.
2. **Non-Preemptive Scheduling** – A process retains the CPU until it terminates or voluntarily releases it (e.g., FCFS, SJF).
    - pros: Simpler implementation, less overhead.
    - cons: May lead to longer waiting times for high-priority processes.

3 Understanding Scheduling Algorithms

3.1 First Come First Serve (FCFS)

FCFS embodies the simplicity and impartiality, allocating the CPU to processes in the exact order they enter the ready queue. No process is favored, no matter its urgency or brevity; it is a paradigm of chronological fairness, where the first to arrive is the first to be served. This non-preemptive approach ensures that once a process claims the CPU, it runs to completion, undisturbed by latecomers.

3.1.1 Diagram



3.1.2 Pros

1. Unadorned Simplicity: FCFS is elegantly straightforward, requiring minimal computational overhead to maintain a first-in, first-out queue.
2. Equitable in Sequence: It upholds a sense of justice by honoring arrival order, ensuring no process is overlooked.
3. Predictable Execution: The deterministic nature makes it easy to anticipate process completion times.

3.1.3 Cons

1. The Convoy Effect: A protracted process at the head of the queue can delay all subsequent processes, much like a slow-moving cart blocking a narrow road.
2. Inefficient for Interactive Systems: Its rigid structure is ill-suited for environments demanding rapid response times, as short processes languish behind lengthy ones.
3. Suboptimal Waiting Times: The average waiting time can balloon up when process burst times vary significantly.

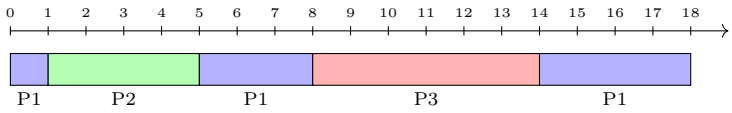
3.1.4 Ideal environments

FCFS flourishes in batch processing systems, where processes are executed sequentially without urgency, such as payroll computations or archival data processing. It is most effective in scenarios with uniform process durations, where the convoy effect is mitigated, and simplicity is paramount. Imagine a library processing book-returns in arrival order, where fairness in sequence trumps speed.

3.2 Shortest Job First (SJF)

SJF operates with a foresight where it selects the process with the shortest burst time from the ready queue. Available in non-preemptive (completing the current process) and preemptive (interrupting for a shorter incoming process, known as Shortest Remaining Time First) variants, SJF seeks to minimize the average waiting time, optimizing throughput with surgical precision.

3.2.1 Diagram



3.2.2 Pros

1. Optimal Efficiency: SJF mathematically minimizes average waiting time in non-preemptive scenarios.
2. Rapid Throughput: By prioritizing brevity, it expedites the completion of short processes, enhancing system responsiveness.

3. Predictable Performance: When burst times are known, SJF delivers consistent, high-efficiency outcomes.

3.2.3 Cons

1. Starvation: Lengthy processes risk indefinite postponement if short processes arrive incessantly, akin to a long order perpetually deferred.
2. Burst Time Dependency: Accurate estimation of execution times is critical, a challenge in dynamic environments.
3. Unsuitable for Interactivity: Long processes endure extended waits, undermining user experience in interactive systems.

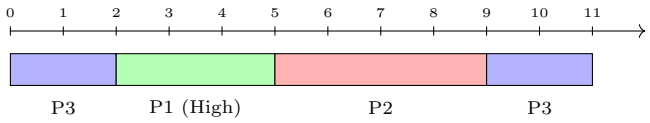
3.2.4 Ideal environments

SJF excels in batch systems with predictable burst times, such as scientific simulations or database transactions, where execution durations can be estimated. It is ideal for environments dominated by short, frequent tasks, ensuring swift completion and high throughput. Imagine a print shop prioritizing quick jobs to keep the queue moving, where efficiency is the currency of success.

3.3 Priority Scheduling

Priority Scheduling mirrors the hierarchy where it assigns each process a priority value (often a lower number for higher priority) and grants the CPU to the highest-priority process. Available in preemptive (evicting lower-priority processes) and non-preemptive forms, it ensures that critical tasks are attended to with utmost urgency, reflecting a system of deliberate precedence.

3.3.1 Diagram



3.3.2 Pros

1. Hierarchical Precision: It ensures critical processes, such as system interrupts, are prioritized, maintaining system integrity.
2. Flexible Customization: Priorities can reflect urgency, deadlines, or resource needs, offering tailored scheduling.
3. Versatile Application: It accommodates mixed workloads, balancing urgent and routine tasks.

3.3.3 Cons

1. Starvation Risk: Low-priority processes may languish indefinitely if high-priority tasks dominate, requiring mechanisms like aging to mitigate.
2. Increased Complexity: Managing and updating priorities introduces computational overhead.
3. Potential Unfairness: Without careful tuning, the system may favor certain processes excessively.

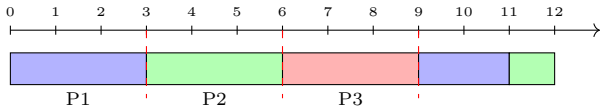
3.3.4 Ideal environments

Priority Scheduling shines in real-time systems, where time-critical tasks—like medical equipment controls or avionics—demand immediate attention. It is also suited for general-purpose operating systems, prioritizing kernel tasks over user applications. For example, envision a hospital triage unit, where emergencies are addressed first, ensuring life-saving interventions take precedence.

3.4 Round Robin (RR)

Round Robin embodies a democratic ethos, allocating the CPU to each process for a fixed time quantum before cycling to the next in a circular queue. This preemptive approach fosters fairness and responsiveness, making it the cornerstone of time-sharing systems where no process is left unheard.

3.4.1 Diagram



3.4.2 Pros

1. Equitable Access: Every process receives regular CPU attention, preventing monopolization.
2. Responsive Design: Frequent context switches ensure quick responses, ideal for interactive systems.
3. Starvation-Free: All processes progress, albeit incrementally, ensuring fairness.

3.4.3 Cons

1. Context Switching Overhead: Frequent transitions between processes consume CPU cycles, reducing efficiency.
2. Quantum Sensitivity: An ill-chosen quantum, too short or too long, can degrade performance, mimicking FCFS or excessive switching.
3. Delayed Completion for Long Jobs: The long processes are fragmented, prolonging their completion.

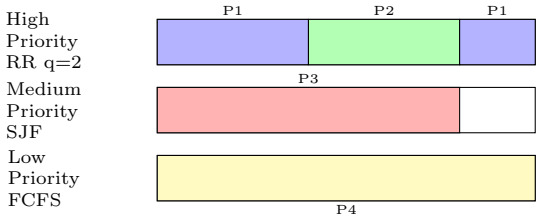
3.4.4 Ideal environments

RR thrives in time-sharing systems, such as modern operating systems (Linux, Windows), where multiple users or applications demand concurrent attention. It is perfect for interactive environments, like web servers or graphical interfaces, where responsiveness is paramount.

3.5 Multilevel Queue

Multilevel Queue scheduling organizes processes into separate queues based on their type or priority, each governed by its own algorithm (e.g., RR for interactive tasks, FCFS for batch jobs). Higher-priority queues are served first, creating a structured hierarchy that balances diverse workloads.

3.5.1 Diagram



3.5.2 Pros

1. Tailored Efficiency: Each queue employs an algorithm suited to its process type, optimizing performance.
2. Clear Prioritization: Critical tasks in higher queues receive prompt attention, ensuring system stability.
3. Structured Organization: Segregating processes by type simplifies management of complex workloads.

3.5.3 Cons

1. Starvation in Lower Queues: Low-priority queues may be neglected if higher queues are perpetually active.
2. Static Assignment: Processes are confined to their initial queue, lacking flexibility to adapt to changing needs.
3. Administrative Complexity: Managing multiple queues and algorithms demands careful configuration.

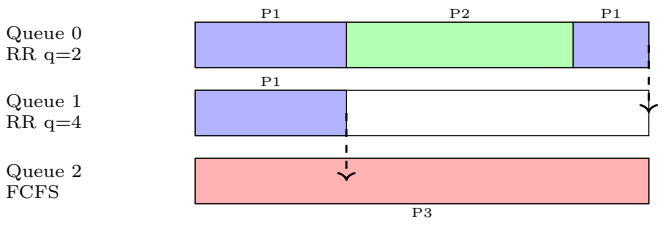
3.5.4 Ideal environments

Multilevel Queue excels in mixed-workload systems, such as servers handling system tasks, user applications, and background jobs. It is ideal for embedded systems or environments with well-defined process categories, where specific tasks require guaranteed priority. Imagine an airport with dedicated lanes for first-class passengers, economy travelers, and cargo, each managed to optimize flow.

3.6 Multilevel Feedback-Queue

Multilevel Feedback Queue scheduling assigns processes to high-priority queues with short quanta and demoting them to lower queues with longer quanta or different algorithms if they consume excessive CPU time. This preemptive, dynamic approach learns from process behavior, favoring short, interactive tasks while ensuring long processes eventually run.

3.6.1 Diagram



3.6.2 Pros

1. Adaptive Intelligence: It dynamically adjusts to process behavior, prioritizing short or interactive tasks while accommodating longer ones.

2. Responsive and Fair: Interactive processes remain in high-priority queues, ensuring quick responses, while no process is permanently sidelined.

3. Balanced Performance: It harmonizes responsiveness, fairness, and efficiency across diverse workloads.

3.6.3 Cons

1. Operational Complexity: Tracking process behavior and managing queue transitions introduce significant overhead.

2. Tuning Challenges: Selecting appropriate quanta and queue rules requires careful calibration to avoid inefficiencies.

3. Resource Intensity: The dynamic nature demands more computational resources than simpler algorithms.

3.6.4 Ideal environments

Multilevel Feedback Queue is the backbone of modern operating systems, such as Linux and Windows, where diverse workloads — system tasks, user applications, and background processes — require balanced scheduling. It shines in dynamic environments with unpredictable process behavior, adapting to ensure responsiveness and fairness.

Each scheduling algorithm is a unique orchestration of priorities, fairness, and efficiency, tailored to the symphony of system demands.

4 Algorithm Performance Evaluation

My chosen 3 algorithms: RR, MLQ and MLFQ.

Hypothetical Process Data:

- **P1:** Arrival Time = 0, Burst Time = 8
- **P2:** Arrival Time = 1, Burst Time = 4
- **P3:** Arrival Time = 2, Burst Time = 6
- **P4:** Arrival Time = 3, Burst Time = 3
- **P5:** Arrival Time = 4, Burst Time = 5

4.1 RR

Table 1: Round Robin Scheduling Timeline (Quantum = 2)

Time Interval	Process
0–2	P1
2–4	P2
4–6	P3
6–8	P4
8–10	P1
10–12	P2
12–14	P3
14–16	P1
16–18	P5
18–20	P3
20–22	P1
22–24	P5
24–26	P5

- t=0-2: P1 (6 units remain).
- t=2-4: P2 (2 units remain).
- t=4-6: P3 (4 units remain).
- t=6-8: P4 (1 unit remain).
- t=8-10: P1 (4 units remain).
- t=10-12: P2 (completes at t=12).
- t=12-14: P3 (2 units remain).
- t=14-16: P1 (2 units remain).
- t=16-18: P5 (3 units remain).
- t=18-20: P3 (completes at t=20).
- t=20-22: P1 (completes at t=22).
- t=22-24: P5 (1 unit remain).
- t=24-26: P5 (completes at t=26).

Metrics

- **Completion times:** P1=22, P2=12, P3=20, P4=8, P5=26.
- **Turnaround Time** = Completion Time - Arrival Time:
  - P1: 22 - 0 = 22
  - P2: 12 - 1 = 11
  - P3: 20 - 2 = 18
  - P4: 8 - 3 = 5
  - P5: 26 - 4 = 22
  - **Average TAT:** (22 + 11 + 18 + 5 + 22) / 5 = 78 / 5 = 15.6
- **Waiting Time** = Turnaround Time - Burst Time:
  - P1: 22 - 8 = 14
  - P2: 11 - 4 = 7
  - P3: 18 - 6 = 12
  - P4: 5 - 3 = 2
  - P5: 22 - 5 = 17
  - **Average WT:** (14 + 7 + 12 + 2 + 17) / 5 = 52 / 5 = 10.4
- **CPU Utilization:** Total Burst Time = 8 + 4 + 6 + 3 + 5 = 26. Schedule Time = 26. Utilization = (26 / 26) * 100 = 100 % (no idle time).
- **Fairness:** High. RR ensures every process gets CPU time in regular intervals, preventing starvation. However, long processes (e.g., P1) are fragmented, which may delay their completion but ensures equitable access.

4.2 MLQ

Configuration:

- Q1: High-priority, system processes (RR, quantum=1). Assume P1 (system process).
- Q2: Medium-priority, interactive processes (RR, quantum=2). P2, P3, P4.
- Q3: Low-priority, batch processes (FCFS). P5.
- Priority: Q1 >Q2 >Q3.

Table 2: Multilevel Queue Scheduling Timeline

Time Interval	Process
0–1	P1
1–2	P1
2–3	P1
3–4	P1
4–5	P1
5–6	P1
6–7	P1
7–8	P1
8–10	P2
10–12	P3
12–14	P4
14–16	P2
16–18	P3
18–19	P4
19–21	P3
21–26	P5

- t=0-8: P1 (Q1, RR quantum=1) runs 8 units (1 unit per cycle), completes at t=8.
- t=8-10: P2 (Q2, RR quantum=2) runs 2 units (2 remain).
- t=10-12: P3 (Q2) runs 2 units (4 remain).
- t=12-14: P4 (Q2) runs 2 units (1 remain).
- t=14-16: P2 runs 2 units, completes at t=16.
- t=16-18: P3 runs 2 units (2 remain).
- t=18-19: P4 runs 1 unit, completes at t=19.
- t=19-21: P3 runs 2 units, completes at t=21.
- t=21-26: P5 (Q3, FCFS) runs 5 units, completes at t=26.

Metrics

- **Completion times:** P1=8, P2=16, P3=21, P4=19, P5=26.
- **Turnaround Time** = Completion Time - Arrival Time:
  - P1: 8 - 0 = 8
  - P2: 16 - 1 = 15
  - P3: 21 - 2 = 19
  - P4: 19 - 3 = 16
  - P5: 26 - 4 = 22
  - **Average TAT:** (8 + 15 + 19 + 16 + 22) / 5 = 80 / 5 = 16
- **Waiting Time** = Turnaround Time - Burst Time:
  - P1: 8 - 8 = 0
  - P2: 15 - 4 = 11
  - P3: 19 - 6 = 13

  - P4: 16 - 3 = 13
  - P5: 22 - 5 = 17
  - **Average WT:** (0 + 11 + 13 + 13 + 17) / 5 = 54 / 5 = 10.8
- **CPU Utilization:** Total Burst Time = 26, Schedule Time = 26.
  Utilization = (26 / 26) * 100 = 100%.
- **Fairness:** Moderate. Q1 processes (P1) get immediate attention, Q2 processes (P2, P3, P4) share CPU fairly via RR, but Q3 (P5) suffers long delays due to lower priority, risking starvation if higher queues are busy.

4.3 MLFQ

Configuration:

- Q1: RR, quantum=2, highest priority.
- Q2: RR, quantum=4, medium priority.
- Q3: FCFS, lowest priority.
- Processes start in Q1, demoted to Q2 after using Q1's quantum, then to Q3 after Q2's quantum.

Table 3: Multilevel Feedback Queue Scheduling Timeline

Time Interval	Process	Queue
0–2	P1	Q1
2–4	P2	Q1
4–6	P3	Q1
6–8	P4	Q1
8–10	P1	Q2
10–12	P2	Q2
12–14	P3	Q2
14–16	P1	Q2
16–18	P5	Q2
18–19	P4	Q2
19–20	P5	Q2
20–21	P1	Q3
21–22	P3	Q3
22–23	P5	Q3
23–24	P1	Q3
24–25	P5	Q3
25–26	P1	Q3

- t=0-2: P1 in Q1 (6 remain), demoted to Q2.
- t=2-4: P2 in Q1 (2 remain), demoted to Q2.
- t=4-6: P3 in Q1 (4 remain), demoted to Q2.
- t=6-8: P4 in Q1 (1 remain), demoted to Q2.
- t=8-10: P1 in Q2 (4 remain), demoted to Q3.
- t=10-12: P2 in Q2, completes at t=12.
- t=12-14: P3 in Q2 (2 remain), demoted to Q3.

- t=14-16: P1 in Q2 (2 remain), demoted to Q3.
- t=16-18: P5 in Q2 (3 remain), demoted to Q3.
- t=18-19: P4 in Q2, completes at t=19.
- t=19-20: P5 in Q2 (2 remain), demoted to Q3.
- t=20-21: P1 in Q3, runs 1 unit (1 remain).
- t=21-22: P3 in Q3, completes at t=22.
- t=22-23: P5 in Q3, completes at t=23.
- t=23-24: P1 in Q3, completes at t=24.
- t=24-26: P5 in Q3 (remaining 2 units), completes at t=26.

Metrics

- **Completion times:** P1=24, P2=12, P3=22, P4=19, P5=26.
- **Turnaround Time** = Completion Time - Arrival Time:
  - P1: 24 - 0 = 24
  - P2: 12 - 1 = 11
  - P3: 22 - 2 = 20
  - P4: 19 - 3 = 16
  - P5: 26 - 4 = 22
  - **Average TAT:** (24 + 11 + 20 + 16 + 22) / 5 = 93 / 5 = 18.6
- **Waiting Time** = Turnaround Time - Burst Time:
  - P1: 24 - 8 = 16
  - P2: 11 - 4 = 7
  - P3: 20 - 6 = 14
  - P4: 16 - 3 = 13
  - P5: 22 - 5 = 17
  - **Average WT:** (16 + 7 + 14 + 13 + 17) / 5 = 67 / 5 = 13.4
- **CPU Utilization:** Total Burst Time = 26, Schedule Time = 26.
  Utilization = (26 / 26) * 100 = 100%.
- **Fairness:** High. MLFQ favors short, interactive processes (P2, P4) by keeping them in higher queues, while long processes (P1, P3, P5) are demoted but still complete, preventing starvation through dynamic queue movement.

4.4 Comparative analysis

Table 4: Comparative Analysis of CPU Scheduling Algorithms

Algorithm	Avg. Waiting Time	Avg. Turnaround Time	CPU Utilization	Fairness
Round Robin (RR)	10.4	15.6	100%	High (Equal CPU sharing)
Multilevel Queue (MLQ)	10.8	16.0	100%	Moderate (Low-priority delays)
Multilevel Feedback Queue (MLFQ)	13.4	18.6	100%	High (Adaptive, no starvation)

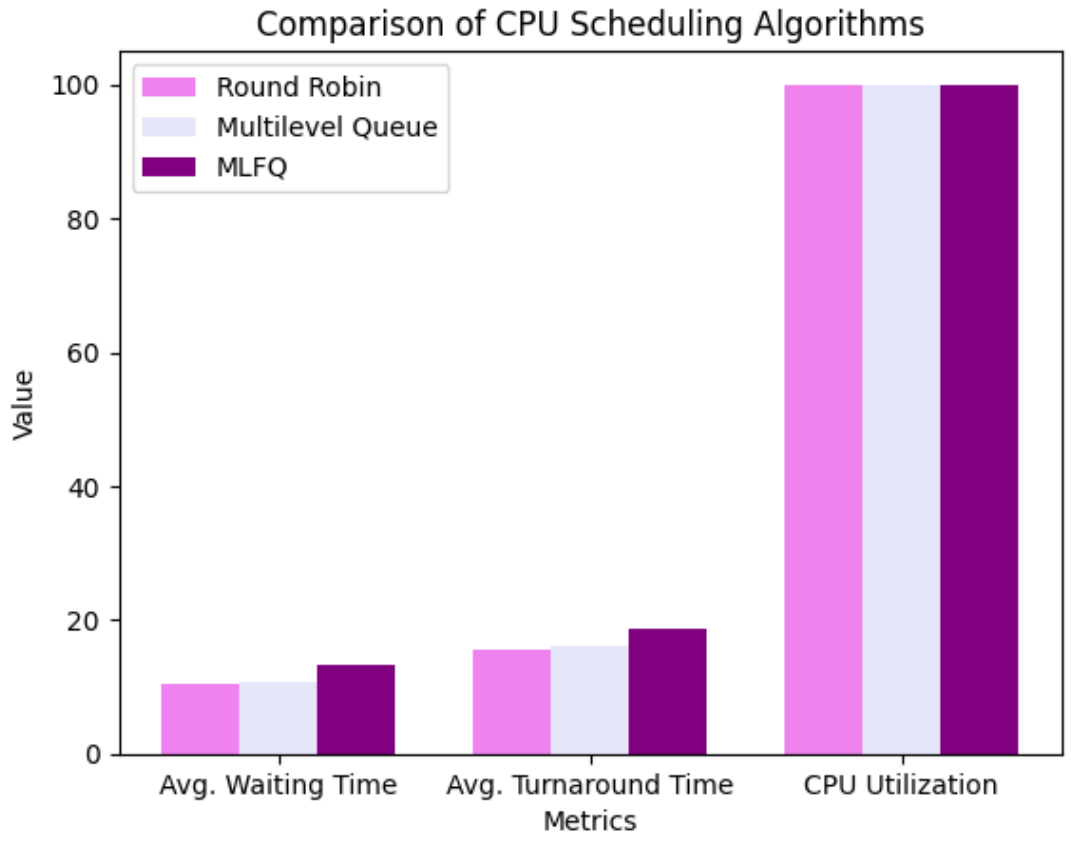


Figure 1: Comparative analysis (google colab)

5 Analytical Application

5.1 How Does Preemption Influence System Responsiveness and Fairness?

Preemption, the act of interrupting a running process to allocate the CPU to another, is akin to a conductor pausing one musician to let another play a more urgent note. Across the six scheduling algorithms, preemption enhances system responsiveness by allowing the CPU to swiftly address newly arrived or higher-priority processes, ensuring that interactive or time-sensitive tasks receive timely attention. For instance, in time-sharing environments, preemption enables rapid context switching, reducing the perception of delay for users interacting with applications. This is particularly vital in systems where responsiveness is paramount, such as modern operating systems or web servers, where users expect near-instantaneous feedback.

However, preemption's impact on fairness is a delicate balance. By prioritizing certain processes (e.g., those with shorter bursts or higher priorities), preemption ensures equitable access for urgent tasks, preventing monopolization by long-running processes. Yet, it can undermine fairness if low-priority or lengthy processes are repeatedly interrupted, delaying their completion. The overhead of frequent context switches also introduces a trade-off, as it consumes CPU cycles that could otherwise be used for computation, potentially affecting overall system efficiency. Across the algorithms, preemption fosters responsiveness but requires careful tuning to maintain fairness, ensuring no process is perpetually sidelined.

5.2 How does starvation occur in scheduling, and which algorithms are prone to it?

Starvation occurs when a process is indefinitely postponed, unable to access the CPU despite being ready, much like a patron at a busy café who never gets served because others keep cutting the line. In CPU scheduling, starvation arises when certain processes are consistently deprioritized due to the scheduling policy. This is often triggered by:

- Priority-Based Selection: Algorithms that favor high-priority or short-burst processes can neglect lower-priority or longer processes if new, favored processes keep arriving.
- Static Queue Assignments: Systems with rigid hierarchies may indefinitely delay processes in lower-priority queues if higher queues remain active.
- Dynamic Workloads: In environments with frequent arrivals of high-priority or short tasks, less favored processes struggle to gain CPU time.

Among the six algorithms, starvation is a risk in those that prioritize specific criteria (e.g., burst time, priority, or queue level) without mechanisms to adjust for prolonged delays. The absence of preemption in some algorithms can exacerbate starvation by allowing a single process to dominate, while excessive preemption in others can cause lower-priority processes to be perpetually interrupted.

5.3 Suggest and justify techniques (like aging) that address starvation.

1. Aging
    - Concept: Aging incrementally increases the priority of a waiting process over time, much like moving a patient up a hospital triage list as their wait extends. This ensures that even low-priority or long processes eventually gain sufficient priority to be scheduled.
    - Justification: Aging is effective because it dynamically adjusts to process wait times, preventing indefinite postponement. It integrates seamlessly into priority-based or multilevel systems, balancing responsiveness with fairness without requiring complex reconfiguration.
2. Priority Boosting
    - Concept: Periodically elevating the priority of processes in lower queues or with long wait times, akin to a teacher calling on quieter students to ensure they contribute.
    - Justification: This technique prevents starvation in hierarchical systems by guaranteeing periodic CPU access, particularly in multilevel setups where lower queues might otherwise be neglected.
3. Time Quantum Adjustments
    - Concept: Allocating larger time quanta to processes that have waited excessively, similar to giving a long-waiting customer a larger serving to compensate.
    - Justification: This approach enhances fairness in time-sharing systems by ensuring delayed processes receive adequate CPU time, reducing the impact of frequent preemption.
4. Queue Rotation

- Concept: Periodically reordering queues or processes to promote those at the back, like rotating a playlist to give every song a chance.
- Justification: This prevents static queue assignments from causing starvation, ensuring all processes cycle through higher-priority states over time.

These methods are robust because they address the root causes of starvation—static prioritization and lack of dynamic adjustment—without compromising system responsiveness. Aging and priority boosting are particularly effective across the six algorithms, as they can be applied to both priority-based and multilevel systems, ensuring fairness while preserving the algorithms' core strengths. Time quantum adjustments and queue rotation complement preemptive systems, enhancing their ability to balance interactivity and equity. By implementing these techniques, the CPU scheduler transforms from a rigid gatekeeper into an adaptive orchestrator, ensuring every process, no matter how humble, eventually takes center stage.

6 Real-World OS Case Study Comparison

6.1 Scheduling Algorithms and Hybrid Models

6.1.1 Linux: The Completely Fair Scheduler (CFS)

Linux employs the Completely Fair Scheduler (CFS), a sophisticated evolution , designed to allocate CPU time equitably across processes, akin to a conductor ensuring every musician plays their part. The CFS operates as a hybrid model, blending elements of Round Robin (RR) with a dynamic, priority-driven approach. It maintains a red-black tree to track processes, ordered by their virtual runtime — a measure of CPU time consumed, adjusted by priority. This structure allows Linux to select the process with the lowest virtual runtime, ensuring fairness while permitting preemption for urgent tasks. For real-time processes, Linux supports SCHED_FIFO (First-In-First-Out) and SCHED_RR (Round Robin with fixed quanta), adhering to POSIX standards for time-critical applications. The CFS's O(1) scheduling complexity, achieved through per-CPU runqueues and array-based priority management, ensures scalability across multiprocessor systems.

6.1.2 Windows: Multilevel Feedback Queue (MLFQ)

Windows, wields a Multilevel Feedback Queue (MLFQ) scheduler, a dynamic and preemptive framework that adapts to process behavior, much like a maestro adjusting tempo to suit the ensemble. This hybrid model integrates priority-based scheduling with Round Robin within each priority level, using 32 priority levels (0–31), divided into real-time (16–31) and variable (1–15) classes, plus a special idle thread at priority 0. Windows dynamically adjusts thread priorities within the variable class, lowering them when a thread exhausts its quantum and boosting them after I/O events (e.g., keyboard input receives a larger boost than disk I/O to enhance interactivity). The MLFQ's adaptability allows Windows to favor short, interactive tasks while ensuring long-running processes progress, making it a versatile conductor for diverse workloads.

6.2 Process Prioritization and Context Switch Management

6.2.1 Linux: Prioritization and Context Switches

Linux prioritizes processes using a dual-range system: real-time priorities (0–99) for critical tasks and a "nice" range (100–140, mapped to -20 to +19) for regular processes, where lower values indicate higher priority. Higher-priority tasks receive longer time quanta, ensuring timely execution, while the CFS adjusts quanta dynamically based on system load and targeted latency—a period during which all runnable tasks should execute at least once. For example, I/O-bound tasks, which frequently yield the CPU, accrue less virtual runtime, gaining implicit priority over CPU-bound tasks, enhancing interactivity.

Context switches in Linux are optimized for efficiency, with the kernel invoking the scheduler at key points (e.g., quantum expiration, I/O completion) to minimize starvation. Each processor maintains its own runqueue, reducing contention in SMP systems, and tasks are stored in active and expired arrays, swapped when the active array empties. This approach, combined with kernel preemption (introduced in Linux 2.5), ensures low dispatch latency, critical for responsiveness. However, frequent context switches can introduce

overhead, particularly with small quanta, which Linux mitigates by balancing quantum size with system load.

6.2.2 Windows: Prioritization and Context Switches

Windows assigns priorities based on process class (e.g., Idle, Normal, High) and relative thread priorities within each class (e.g., Time Critical, Normal), mapped to the 32-level priority scheme. Foreground processes (active windows) receive a quantum multiplier (typically 3x), boosting responsiveness for interactive tasks, such as typing or mouse clicks. After a thread consumes its quantum, its priority may drop (but not below its base), while I/O completion triggers a priority boost, with larger boosts for keyboard events to prioritize user interaction.

Context switches are managed by the Windows dispatcher, which performs rapid switches (saving and restoring Process Control Blocks) to maintain responsiveness. Windows tracks CPU cycles precisely using the processor's cycle counter (since Windows Vista), ensuring accurate quantum enforcement. Context switch overhead is minimized by avoiding unnecessary switches (e.g., not charging interrupt handling to thread quanta) and leveraging CPU affinity to reduce cache thrashing in multiprocessor systems. However, the MLFQ's multiple queues introduce complexity, potentially increasing overhead when switching between priority levels.

6.3 Scheduling Policies: Desktop vs. Server Versions

6.3.1 Linux: Desktop vs. Server

Linux's CFS is consistent across desktop and server environments, reflecting its monolithic kernel design, but tuning parameters differ to suit workload demands. On desktop systems (e.g., Ubuntu Desktop), Linux optimizes for responsiveness, using smaller time quanta (e.g., 6–10 ms) to ensure low-latency interactions for GUI applications, such as video playback or gaming. The scheduler prioritizes I/O-bound tasks, and real-time policies (SCHED_RR, SCHED_FIFO) support multimedia or soft real-time applications.

On **server systems** (e.g., Ubuntu Server, CentOS), Linux emphasizes throughput and fairness, with larger quanta (e.g., 20–50 ms) to reduce context switch overhead for CPU-bound workloads, such as database queries or web serving. The CFS's scalability shines in SMP and NUMA systems, where per-CPU runqueues and processor affinity minimize latency. Server configurations may also tune the "nice" values or use cgroups to allocate CPU shares, ensuring critical services (e.g., Apache, MySQL) receive priority. Real-time scheduling is less common but used for hard real-time tasks in embedded or specialized servers.

6.3.2 Windows: Desktop vs. Server

Windows tailors its MLFQ scheduler distinctly for desktop and server editions, reflecting their divergent priorities. On desktop systems (e.g., Windows 11), the scheduler prioritizes interactivity, aggressively boosting foreground processes with extended quanta and higher priorities to ensure smooth user experiences, such as animations or application launches. The Multimedia Class Scheduler Service (introduced in Vista) supports real-time audio/video playback, adjusting priorities dynamically for low-latency streaming. Smaller quanta (e.g., 10–15 ms) reduce response times but increase context switch frequency, acceptable for interactive workloads.

On **server systems**, the focus shifts to throughput and stability for long-running, CPU-intensive tasks, such as virtualization or database hosting. Windows Server uses larger quanta (e.g., 20–36 ms) to minimize context switches, enhancing efficiency for batch processes. Priority boosts are less aggressive, and background processes receive fairer treatment to prevent starvation in high-load scenarios. CPU affinity is leveraged to optimize cache locality in multiprocessor environments, critical for server performance. Real-time priorities (16–31) are reserved for critical system tasks, with stricter admission controls to ensure stability.

6.4 Analysis and Synthesis

Linux and Windows, like skilled conductors, orchestrate CPU scheduling with distinct philosophies. Linux's CFS, with its red-black tree and virtual runtime, champions fairness and scalability, making it a versatile

choice for both desktop responsiveness and server throughput. Its per-CPU runqueues and dynamic quanta minimize context switch overhead, ideal for multiprocessor systems. Windows' MLFQ, with its tiered priorities and dynamic boosts, excels in interactivity, particularly on desktops, where foreground tasks shine. However, its complex queue management and aggressive boosting can compromise fairness for background tasks, especially on servers.

The desktop-server divide highlights their tailored approaches: Linux adjusts parameters to shift from low-latency desktops to high-throughput servers, while Windows embeds policy differences, favoring interactivity on desktops and stability on servers. Both systems mitigate starvation—Linux through virtual runtime fairness, Windows via priority boosts and aging—but Linux's unified model offers greater tunability, while Windows' structured MLFQ ensures predictable responsiveness for user-facing tasks.

7 Visualization Task

- P1: Arrival Time = 0, Burst Time = 8
- P2: Arrival Time = 1, Burst Time = 4
- P3: Arrival Time = 2, Burst Time = 6
- P4: Arrival Time = 3, Burst Time = 3
- P5: Arrival Time = 4, Burst Time = 5

Total Burst Time: 8 + 4 + 6 + 3 + 5 = 26 units.

Figure 2: Gantt Chart for Round Robin Scheduling (Quantum = 2)

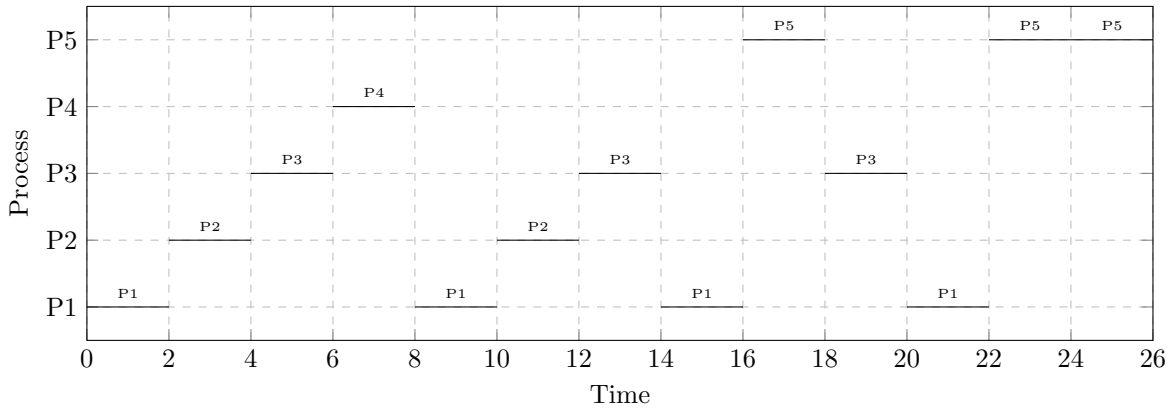
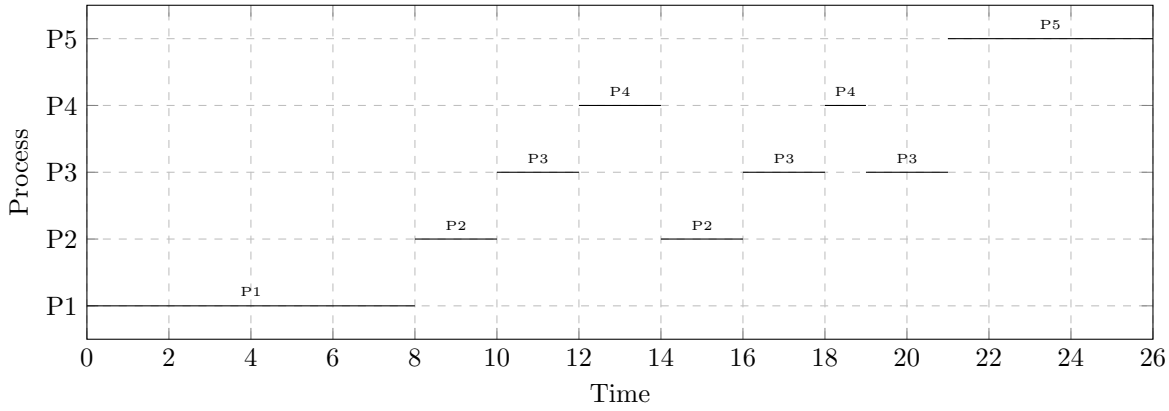


Figure 3: Gantt Chart for Multilevel Queue Scheduling



8 Creative Thinking Scenario

In crafting an operating system for a real-time embedded system, such as a pacemaker or automotive controller, where lives hang in the balance, the choice of CPU scheduling algorithm is paramount. I propose the **Priority Scheduling algorithm**, specifically in its preemptive form, as the cornerstone of this system, justified by its exemplary reliability, determinism, and low latency.

- Reliability: Priority Scheduling ensures that critical tasks, such as heart rhythm adjustments in a pacemaker or brake activation in an automotive controller, are assigned the highest priority and executed without fail. Its preemptive nature guarantees that urgent processes interrupt less critical ones, safeguarding system integrity in life-critical scenarios.
- Determinism: By adhering to a strict priority hierarchy, this algorithm delivers predictable execution, ensuring that high-priority tasks meet stringent deadlines. This determinism is vital for real-time systems, where timing precision is non-negotiable, akin to a conductor ensuring every note is played on cue.
- Latency: Preemptive Priority Scheduling minimizes latency by swiftly reallocating the CPU to high-priority tasks upon their arrival, reducing response times to near-instantaneous levels. This responsiveness is crucial for embedded systems, where delays could prove catastrophic.

While alternatives like RR offer fairness, they lack the deterministic precision required, and Multilevel Feedback Queues introduce complexity unsuitable for resource-constrained environments. Priority Scheduling, with its focused simplicity and preemptive agility, stands as the sentinel of reliability, ensuring that the system's heartbeat—be it literal or vehicular—remains steadfast and true.

9 Conclusion

CPU scheduling algorithms like Round Robin (RR), Multilevel Queue (MLQ), and Multilevel Feedback Queue (MLFQ) balance performance and fairness. RR (10.4 ms wait) offers responsiveness, MLQ (10.8 ms) structures workloads, and MLFQ (13.4 ms) adapts to diverse demands. Preemption improves interactivity but may add overhead; starvation, common in MLQ, is reduced in MLFQ via aging. Modern systems like Linux's CFS and Windows' MLFQ integrate fairness with adaptability, while Priority Scheduling ensures reliability in critical systems.

Reflection: Scheduling drives system performance, balancing user responsiveness, server throughput, and real-time predictability. It's a careful trade-off of fairness and efficiency—a craft that shapes computing's rhythm.